

Optimization with Quantum Computers

Combinatorial optimization means searching for an optimal solution in a finite or countably infinite set of potential solutions. Optimality is defined with respect to some criterion function, to be minimized or maximized, which is typically called the objective function.

This cost function can be mapped to a Hamiltonian that is diagonal in the computational basis. Given the cost-function C this Hamiltonian is then written as:

$$H = \sum_{x \in \{0,1\}^n} C(x) |x\rangle\langle x|$$

where $x \in \{0,1\}^n$ labels the computational basis states $|x\rangle \in \mathbb{C}^{2^n}$.

1 Qiskit Optimization

Qiskit Optimization supports Quadratically Constrained Quadratic Programs:

$$\begin{aligned} & \text{minimize} && x^\top Q_0 x + c^\top x \\ & \text{subject to} && Ax \leq b \\ & && x^\top Q_i x + a_i^\top x \leq r_i, \quad 1, \dots, i, \dots, q \\ & && l_i \leq x_i \leq u_i, \quad 1, \dots, i, \dots, n; \end{aligned}$$

where Q_i are $n \times n$ matrices, A is a $m \times n$ matrix, x , and c are n -dimensional vectors.

However, quantum algorithms can not handle generic Quadratic Programs directly. Quadratic Programs need to be converted into Quadratic Unconstrained Binary Optimization (QUBO) problems, and eventually Ising Hamiltonians.

Suppose a QUBO

$$\min_{x \in \{0,1\}^n} \sum_{i,j=0}^{n-1} A_{ij} x_i x_j + c$$

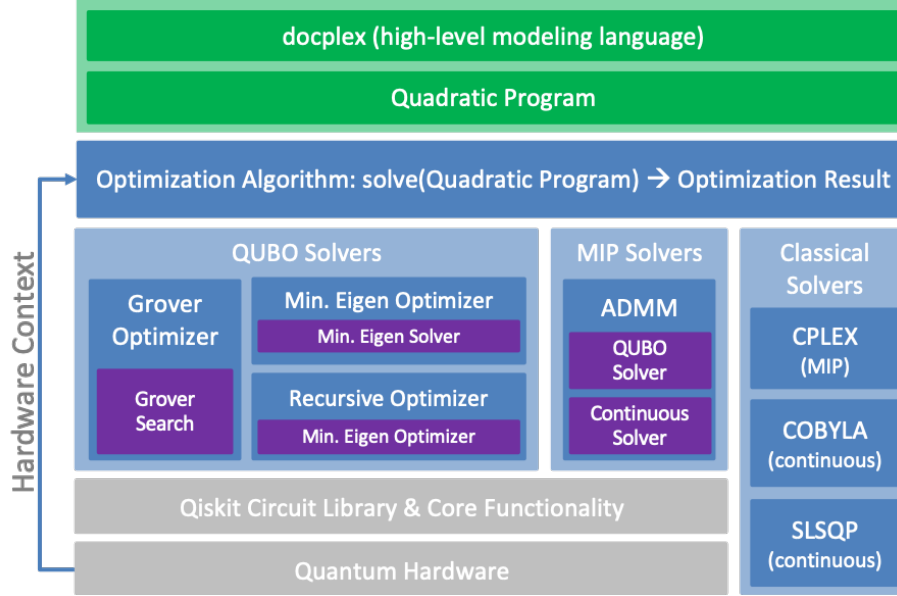
which has been mapped to an Ising Hamiltonian:

$$H = \sum_{i,j=0}^{n-1} \tilde{A}_{ij} \sigma_Z^i \otimes \sigma_Z^j + \sum_{i=0}^{n-1} \tilde{b}_i \sigma_Z^i + \tilde{c}$$

where σ_Z^i denotes the Pauli Z-matrix $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ on the i -th qubit.

Finding the groundstate of h is equivalent to solving the QUBO: $\min_{|\psi\rangle} \langle \psi | H | \psi \rangle$

Qiskit optimization offers automatic conversion of problems to Ising Hamiltonian and allows users to solve problems using a large set of algorithms (QAOA, Grover adaptive Search, classical solvers, etc.)¹.



2 Initial Assignment

The student should study and acquire an overall understanding of the principles behind quantum optimization and how one can solve practical optimization problems using the latest qiskit optimization module.

3 Final Assignment

One of the disadvantages of operating in the QUBO domain, is that inequality constraints need to be converted to equality ones, introducing an overhead in terms of slack variables, i.e. qubits needed for the computation. Moreover, we are bound to use only binary variables, which may not be the case in real world scenarios and applications.

Among others, the *ADMM Optimizer* [1] can solve classes of mixed-binary constrained optimization problems, hereafter (MBCO), which often appear in

¹<https://qiskit.org/documentation/optimization/tutorials/index.html>

logistic, finance, and operation research. In particular, the ADMM Optimizer implemented in Qiskit²³ can tackle the following optimization problem:

$$\min_{x \in \mathcal{X}, u \in \mathcal{U} \subseteq \mathbb{R}^l} q(x) + \varphi(u) \quad (1)$$

subject to the constraints:

$$\text{s.t. : } \quad Gx = b, \quad g(x) \leq 0, \quad \ell(x, u) \leq 0, \quad (2)$$

with the corresponding functional assumptions.

- Function $q : \mathbb{R}^n \rightarrow \mathbb{R}$ is quadratic, i.e., $q(x) = x^\top Qx + a^\top x$ for a given symmetric squared matrix $Q \in \mathbb{R}^n \times \mathbb{R}^n, Q = Q^\top$, and vector $a \in \mathbb{R}^n$,
- The set $\mathcal{X} = \{0, 1\}^n = \{x_{(i)}(1 - x_{(i)}) = 0, \forall i\}$ enforces the binary constraints,
- Matrix $G \in \mathbb{R}^n \times \mathbb{R}^{n'}$, vector $b \in \mathbb{R}^{n'}$, and function $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex,
- Function $\varphi : \mathbb{R}^l \rightarrow \mathbb{R}$ is convex and $\mathcal{U}$ is a convex set,
- Function $\ell : \mathbb{R}^n \times \mathbb{R}^l \rightarrow \mathbb{R}$ is jointly convex in x, u .

The method leverages the ADMM operator-splitting procedure to devise a decomposition for certain classes of MBOs into:

- a QUBO sub-problem to be solved by on the quantum device via variational algorithms, such as VQE or QAOA;
- a continuous convex constrained sub-problem, which can be efficiently solved with classical optimization solvers.

3.1 Problem of Interest

Consider the problem of allocating virtual machines to servers in order to optimize the power consumption, while also not overloading servers

$$\begin{aligned} \min \quad & \sum_{i=1}^M \left(P_i^I s_i + P_i^D \sum_{j=1}^N u(v_j) v_{ji} \right) \\ \text{s.t.} \quad & \sum_{j=0}^N u(v_j) v_{ji} \leq C_i s_i, \forall i \in [1, M] \\ & \sum_{i=1}^M v_{ji} = 1, \forall j \in [1, N] \end{aligned}$$

²https://qiskit.org/documentation/optimization/tutorials/05_admm_optimizer.html

³https://github.com/Qiskit/qiskit-optimization/blob/main/qiskit_optimization/algorithms/admm_optimizer.py

where:

- $s_i \in \{0, 1\}$ indicates if the i -th server is on/off
- $v_{ji} \in \{0, 1\}$ assigns the j -th virtual machine to the i -th server
- $P_i^I \in \mathbb{N}$ is the idle power consumption when server s_i is on
- $P_i^D \in \mathbb{N}$ is the dynamic power consumption of server s_i
- C_i is the maximum CPU capacity of server s_i
- $u(v_j) \in \mathbb{N}$, $0 < u(v_j) \leq C_i$ is the CPU load of the j -th virtual machine

The student should investigate the potential application of the ADMM Optimizer to such a problem. It would be especially interesting to determine the dimension of the QUBO and the continuous sub-problem. To do this, it would probably be necessary to look into the Qiskit implementation and extract the sub-problems objects.

References

- [1] C. Gambella and A. Simonetto, "Multiblock ADMM Heuristics for Mixed-Binary Optimization on Classical and Quantum Computers," in IEEE Transactions on Quantum Engineering, vol. 1, pp. 1-22, 2020, Art no. 3102022, doi: 10.1109/TQE.2020.3033139.